```
import { useState, useEffect, useRef, useCallback } from "react";

/* ═══════════════════════════════════════════════════════════
   NOTIFICATION ENGINE
   - 브라우저 Notification API (백그라운드 포함)
   - 인앱 토스트 배너 (앱 안에 있을 때)
   - 일정 start 시간에 맞춰 자동 스케줄링
═══════════════════════════════════════════════════════════ */

// 전역 알림 토스트 이벤트 버스
const toastBus = { listeners: [], emit(msg) { this.listeners.forEach(fn => fn(msg

function scheduleTaskNotifications(tasks, dateKey) {
  // 오늘 날짜인지 확인
  const todayStr = (() => { const n=new Date(); return `${n.getFullYear()}-${Stri
  if (dateKey !== todayStr) return [];

  const timers = [];
  const now = new Date();

  tasks.forEach(task => {
    if (!task.start || task.done) return;
    const [h, m] = task.start.split(":").map(Number);
    const target = new Date();
    target.setHours(h, m, 0, 0);

    // 5분 전 알림
    const before5 = new Date(target.getTime() - 5 * 60 * 1000);
    const msB5 = before5 - now;
    if (msB5 > 0) {
      const t = setTimeout(() => {
        const msg = { id: Date.now(), icon: "⏰", title: "5분 후 시작", body: `"${t
        toastBus.emit(msg);
        if (Notification.permission === "granted") {
          new Notification(`⏰ 5분 후 — ${task.text}`, { body: `${task.start} 시작
        }
      }, msB5);
      timers.push(t);
    }

    // 정시 알림
    const msExact = target - now;
    if (msExact > 0 && msExact < 24 * 60 * 60 * 1000) {
      const t = setTimeout(() => {
```

```
        const catIcon = { 필수:"⭐", 휴식:"🌿", 루틴:"🔄", 작업:"💼" }[task.category]
        const msg = { id: Date.now(), icon: catIcon, title: "지금 시작!", body: `"$
        toastBus.emit(msg);
        if (Notification.permission === "granted") {
          new Notification(`${catIcon} 지금 시작 — ${task.text}`, { body: `${task.s
        }
      }, msExact);
      timers.push(t);
    }
  });

  return timers;
}

/* 인앱 토스트 배너 */
function ToastBanner({ C }) {
  const [toasts, setToasts] = useState([]);

  useEffect(() => {
    const handler = msg => {
      setToasts(prev => [...prev.slice(-2), { ...msg, key: Date.now() + Math.rand
    };
    toastBus.listeners.push(handler);
    return () => { toastBus.listeners = toastBus.listeners.filter(f => f !== hand
  }, []);

  const dismiss = key => setToasts(prev => prev.filter(t => t.key !== key));

  useEffect(() => {
    if (toasts.length === 0) return;
    const latest = toasts[toasts.length - 1];
    const t = setTimeout(() => dismiss(latest.key), 5000);
    return () => clearTimeout(t);
  }, [toasts]);

  if (toasts.length === 0) return null;

  return (
    <div style={{ position:"fixed", top:72, left:0, right:0, zIndex:500,
      display:"flex", flexDirection:"column", alignItems:"center", gap:8, pointer
      <style>{`
        @keyframes toastIn { from { opacity:0; transform:translateY(-16px) scale(
        @keyframes toastOut { from { opacity:1; } to { opacity:0; transform:trans
      `}</style>
      {toasts.map(toast => (
        <div key={toast.key} style={{
          pointerEvents:"all",
```

```
          background: C.isDark ? "#1e1e2e" : "#ffffff",
          border:`1.5px solid ${toast.color || C.accent}44`,
          borderLeft:`4px solid ${toast.color || C.accent}`,
          borderRadius:16, padding:"12px 16px",
          maxWidth:380, width:"calc(100% - 32px)",
          display:"flex", alignItems:"center", gap:12,
          boxShadow:`0 8px 32px rgba(0,0,0,${C.isDark?".5":".14"})`,
          animation:"toastIn .3s cubic-bezier(.34,1.56,.64,1)",
          cursor:"pointer",
        }} onClick={() => dismiss(toast.key)}>
          <span style={{ fontSize:22, flexShrink:0 }}>{toast.icon}</span>
          <div style={{ flex:1, minWidth:0 }}>
            <p style={{ margin:0, fontSize:13, fontWeight:800, color: toast.color
            <p style={{ margin:"2px 0 0", fontSize:12, color:C.soft,
              overflow:"hidden", textOverflow:"ellipsis", whiteSpace:"nowrap" }}>
          </div>
          <button style={{ background:"none", border:"none", color:C.muted,
            cursor:"pointer", fontSize:18, padding:0, flexShrink:0, lineHeight:1
        </div>
      ))}
    </div>
  );
}

/* 알림 권한 요청 배너 */
function NotifPermBanner({ C, onGrant }) {
  const [show, setShow] = useState(false);

  useEffect(() => {
    if (typeof Notification !== "undefined" && Notification.permission === "defau
      const t = setTimeout(() => setShow(true), 1500);
      return () => clearTimeout(t);
    }
  }, []);

  if (!show) return null;

  const request = async () => {
    const perm = await Notification.requestPermission();
    setShow(false);
    if (perm === "granted") onGrant();
  };

  return (
    <div style={{
      margin:"0 0 14px",
      background: C.isDark ? "#1a1a2e" : "#f0eeff",
```

```
      border:`1.5px solid ${C.accent}55`,
      borderRadius:16, padding:"14px 16px",
      display:"flex", alignItems:"center", gap:12,
    }}>
      <span style={{ fontSize:24 }}>🔔</span>
      <div style={{ flex:1 }}>
        <p style={{ margin:0, fontSize:13, fontWeight:800, color:C.text }}>알림 허용
        <p style={{ margin:"2px 0 0", fontSize:11, color:C.muted }}>일정 시간에 맞춰
      </div>
      <div style={{ display:"flex", gap:6, flexShrink:0 }}>
        <button onClick={()=>setShow(false)} style={{
          background:"none", border:`1px solid ${C.border}`, borderRadius:10,
          padding:"6px 10px", fontSize:11, color:C.muted, cursor:"pointer", fontF
        }}>나중에</button>
        <button onClick={request} style={{
          background:C.accent, border:"none", borderRadius:10,
          padding:"6px 12px", fontSize:11, fontWeight:800, color:"#fff", cursor:"
        }}>허용</button>
      </div>
    </div>
  );
}

/* ═══════════════════════════════════════════════════════════
   THEME
═══════════════════════════════════════════════════════════ */
const LIGHT = {
  bg:"#f5f4fe", surface:"#ffffff", card:"#ffffff",
  border:"#e8e5f8", borderLit:"#cbc7ef",
  accent:"#6c5fff", accentDim:"#edeaff", accentGlow:"rgba(108,95,255,.16)",
  green:"#22c47a", greenDim:"#e1f8ee",
  orange:"#f5862a", orangeDim:"#fff0e4",
  red:"#e8334a", redDim:"#fde8ec",
  teal:"#15b8c8", tealDim:"#e0f8fb",
  yellow:"#d4a000", yellowDim:"#fff8e0",
  text:"#1a1730", soft:"#5a5575", muted:"#9c98b8",
  inputBg:"#f8f7ff", shadow:"rgba(80,60,180,.10)",
  navBg:"rgba(255,255,255,.94)", isDark:false,
};
const DARK = {
  bg:"#0d0d12", surface:"#14141c", card:"#1c1c28",
  border:"#2a2a3e", borderLit:"#3a3a54",
  accent:"#7c6fff", accentDim:"#28255a", accentGlow:"rgba(124,111,255,.22)",
  green:"#44d492", greenDim:"#122b1e",
  orange:"#ff9f56", orangeDim:"#2e1c08",
  red:"#ff5c72", redDim:"#2e1018",
  teal:"#3dd8e8", tealDim:"#0c2830",
```

```
  yellow:"#ffd166", yellowDim:"#2a2208",
  text:"#eceaf8", soft:"#a8a4c8", muted:"#58546e",
  inputBg:"#10101a", shadow:"rgba(0,0,0,.45)",
  navBg:"rgba(18,18,28,.95)", isDark:true,
};

const pad    = n => String(n).padStart(2,"0");
const fmtSec = s => `${pad(Math.floor(s/60))}:${pad(s%60)}`;
const dkey   = d => `${d.getFullYear()}-${pad(d.getMonth()+1)}-${pad(d.getDate())
const nowKey = () => dkey(new Date());

/* 달력용: 날짜 달성률 → 색상 */
function completionColor(pct, C) {
  if (pct === 0) return "transparent";
  if (pct < 0.34) return C.orange + "55";
  if (pct < 0.67) return C.yellow + "88";
  if (pct < 1.0)  return C.green  + "88";
  return C.green;
}

/* ═══════════════════════════════════════════════════════════
   DRAG NUMBER
═══════════════════════════════════════════════════════════ */
function DragNum({ value, onChange, min=0, max=999, step=1, unit="", size=52, col
  const sY=useRef(null), sV=useRef(null), drag=useRef(false);
  const [active,setActive]=useState(false);
  const onStart=e=>{ sY.current=e.touches?e.touches[0].clientY:e.clientY; sV.curr
  const onMove=useCallback(e=>{ if(!drag.current)return; const y=e.touches?e.touc
  const onEnd=()=>{ drag.current=false; setActive(false); };
  useEffect(()=>{ window.addEventListener("mousemove",onMove); window.addEventLis
  return (
    <div onMouseDown={onStart} onTouchStart={onStart}
      style={{ display:"flex",flexDirection:"column",alignItems:"center",gap:3,cu
      <span style={{ fontSize:10,color:active?color:"#bbb",fontWeight:700 }}>▲</s
      <span style={{ fontSize:size,fontWeight:900,fontFamily:"'DM Mono','SF Mono'
      {unit&&<span style={{ fontSize:10,color:"#bbb",letterSpacing:1.5 }}>{unit}<
      <span style={{ fontSize:10,color:active?color:"#bbb",fontWeight:700 }}>▼</s
    </div>
  );
}

/* ═══════════════════════════════════════════════════════════
   BOTTOM SHEET
═══════════════════════════════════════════════════════════ */
function Sheet({ show, onClose, children, C, title, height="auto" }) {
  if (!show) return null;
  return (
```

```
    <div style={{ position:"fixed",inset:0,zIndex:200,background:"rgba(0,0,0,.45)
      onClick={onClose}>
      <div style={{ background:C.surface,borderRadius:"24px 24px 0 0",width:"100%
        maxHeight:"90vh",overflowY:"auto",border:`1px solid ${C.border}`,paddingB
        onClick={e=>e.stopPropagation()}>
        <div style={{ width:44,height:5,background:C.border,borderRadius:3,margin
        {title&&<p style={{ fontSize:15,fontWeight:800,color:C.text,margin:"14px
        <div style={{ padding:"14px 20px 0" }}>{children}</div>
      </div>
    </div>
  );
}

/* ═══════════════════════════════════════════════════════════
   일정 등록 바텀시트
═══════════════════════════════════════════════════════════ */
const TASK_TEMPLATES = [
  { icon:"💤", label:"기상",      start:"07:00", end:"07:30", category:"필수" },
  { icon:"🍳", label:"아침 식사",start:"07:30", end:"08:00", category:"필수" },
  { icon:"💊", label:"약 복용",  start:"08:00", end:"08:05", category:"필수" },
  { icon:"🚶", label:"산책",      start:"08:00", end:"08:30", category:"필수" },
  { icon:"☕", label:"커피/차",  start:"09:00", end:"09:15", category:"휴식" },
  { icon:"🍱", label:"점심 식사",start:"12:00", end:"13:00", category:"필수" },
  { icon:"💤", label:"낮잠",      start:"13:00", end:"13:20", category:"휴식" },
  { icon:"🧘", label:"명상",      start:"14:00", end:"14:15", category:"휴식" },
  { icon:"🍽", label:"저녁 식사",start:"18:00", end:"19:00", category:"필수" },
  { icon:"🛁", label:"샤워",      start:"21:00", end:"21:30", category:"필수" },
  { icon:"📚", label:"독서",      start:"22:00", end:"22:30", category:"루틴" },
  { icon:"😴", label:"취침",      start:"23:00", end:"07:00", category:"필수" },
];

const CAT_COLORS = (C) => ({
  필수: C.green,
  휴식: C.teal,
  루틴: C.accent,
  작업: C.orange,
});

function AddTaskSheet({ show, onClose, onAdd, C, defaultDate }) {
  const [text,    setText]    = useState("");
  const [start,   setStart]   = useState("");
  const [end,     setEnd]     = useState("");
  const [actions, setActions]= useState([]);
  const [selCat,  setSelCat] = useState("작업");
  const [selTpl,  setSelTpl] = useState(null);

  const reset = () => { setText(""); setStart(""); setEnd(""); setActions([]); se
```

```
const applyTpl = (t) => {
  setText(t.label); setStart(t.start); setEnd(t.end); setSelCat(t.category); se
};

const addAction = () => setActions(a=>[...a,{id:Date.now(),text:""}]);
const updAction = (id,v) => setActions(a=>a.map(x=>x.id===id?{...x,text:v}:x));
const delAction = id => setActions(a=>a.filter(x=>x.id!==id));

const submit = () => {
  if (!text.trim()) return;
  onAdd({ text:text.trim(), start, end, category:selCat,
    actions:actions.filter(a=>a.text.trim()).map(a=>({...a,done:false})),
    done:false, id:Date.now() });
  reset(); onClose();
};

const catCol = CAT_COLORS(C);
const cats = Object.keys(catCol);

return (
  <Sheet show={show} onClose={()=>{ reset(); onClose(); }} C={C} title="📋 일정
    {/* 빠른 템플릿 */}
    <p style={{ fontSize:11,color:C.muted,fontWeight:700,letterSpacing:.5,margi
    <div style={{ display:"flex",gap:6,overflowX:"auto",paddingBottom:8,marginB
      {TASK_TEMPLATES.map(t=>(
        <button key={t.label} onClick={()=>applyTpl(t)} style={{
          flexShrink:0,display:"flex",flexDirection:"column",alignItems:"center
          padding:"8px 10px",borderRadius:14,border:`1.5px solid ${selTpl===t.l
          background:selTpl===t.label?catCol[t.category]+"22":C.surface,
          cursor:"pointer",fontFamily:"inherit",transition:"all .15s",minWidth:
        }}>
          <span style={{ fontSize:20 }}>{t.icon}</span>
          <span style={{ fontSize:10,fontWeight:700,color:selTpl===t.label?catC
        </button>
      ))}
    </div>

    {/* 제목 */}
    <input value={text} onChange={e=>setText(e.target.value)} placeholder="일정
      autoFocus
      style={{ width:"100%",background:C.inputBg,border:`1.5px solid ${C.border
        borderRadius:14,padding:"13px 16px",color:C.text,fontSize:15,fontFamily
        outline:"none",marginBottom:12 }}/>

    {/* 카테고리 */}
    <div style={{ display:"flex",gap:6,marginBottom:14 }}>
```

```
  {cats.map(c=>(
    <button key={c} onClick={()=>setSelCat(c)} style={{
      flex:1,padding:"8px 4px",borderRadius:12,
      background:selCat===c?catCol[c]+"22":C.surface,
      color:selCat===c?catCol[c]:C.muted,
      border:`1.5px solid ${selCat===c?catCol[c]:C.border}`,
      fontSize:11,fontWeight:800,cursor:"pointer",fontFamily:"inherit",tran
    }}>{c}</button>
  ))}
</div>

{/* 시간 */}
<div style={{ display:"grid",gridTemplateColumns:"1fr auto 1fr",alignItems:
  <div>
    <p style={{ fontSize:11,color:C.muted,marginBottom:4,fontWeight:600 }}>
    <input type="time" value={start} onChange={e=>setStart(e.target.value)}
      style={{ width:"100%",background:C.inputBg,border:`1.5px solid ${C.bo
        borderRadius:12,padding:"11px 10px",color:C.text,fontSize:14,fontFa
  </div>
  <span style={{ color:C.muted,fontWeight:700,marginTop:16 }}>→</span>
  <div>
    <p style={{ fontSize:11,color:C.muted,marginBottom:4,fontWeight:600 }}>
    <input type="time" value={end} onChange={e=>setEnd(e.target.value)}
      style={{ width:"100%",background:C.inputBg,border:`1.5px solid ${C.bo
        borderRadius:12,padding:"11px 10px",color:C.text,fontSize:14,fontFa
  </div>
</div>

{/* 행동 단계 */}
<div style={{ marginBottom:14 }}>
  {actions.map((a,i)=>(
    <div key={a.id} style={{ display:"flex",alignItems:"center",gap:8,margi
      <span style={{ width:20,height:20,borderRadius:"50%",background:C.acc
        color:C.accent,fontSize:10,fontWeight:800,
        display:"flex",alignItems:"center",justifyContent:"center",flexShri
      <input value={a.text} onChange={e=>updAction(a.id,e.target.value)}
        placeholder={`행동 ${i+1} (5~15분 단위)`}
        style={{ flex:1,background:C.inputBg,border:`1.5px solid ${C.border
          borderRadius:10,padding:"9px 12px",color:C.text,fontSize:13,fontF
      <button onClick={()=>delAction(a.id)}
        style={{ background:"none",border:"none",color:C.muted,cursor:"poin
    </div>
  ))}
  <button onClick={addAction} style={{ width:"100%",background:"none",
    border:`1.5px dashed ${C.border}`,borderRadius:12,padding:"9px",
    color:C.muted,fontSize:12,cursor:"pointer",fontFamily:"inherit" }}>
    + 행동 단계 추가
```

```
        </button>
      </div>

      {/* 힌트 */}
      <div style={{ background:C.accentDim,borderRadius:12,padding:"10px 14px",ma
        <p style={{ fontSize:11,color:C.accent,margin:0,lineHeight:1.7 }}>
          💡 행동을 5~15분 단위로 쪼개면 시작하기 훨씬 쉬워요
        </p>
      </div>

      <button onClick={submit} style={{
        width:"100%",background:C.accent,color:"#fff",border:"none",borderRadius:
        padding:"15px",fontSize:15,fontWeight:900,cursor:"pointer",fontFamily:"in
        boxShadow:`0 4px 20px ${C.accentGlow}`,
      }}>일정 추가하기</button>
    </Sheet>
  );
}

/* ═══════════════════════════════════════════════════════════
   CALENDAR  (달성률 히트맵)
═══════════════════════════════════════════════════════════ */
function CalendarView({ tasksByDate, selDate, onSelect, C }) {
  const now=new Date();
  const [mode,  setMode] =useState("week");
  const [year,  setYear] =useState(now.getFullYear());
  const [month, setMonth]=useState(now.getMonth());
  const [wOff,  setWOff] =useState(0);
  const tk=nowKey();

  const getDayPct = k => {
    const tasks=tasksByDate[k]||[];
    if (!tasks.length) return null;
    const done=tasks.filter(t=>t.done).length;
    return done/tasks.length;
  };

  const weekStart=()=>{ const d=new Date(now); d.setDate(d.getDate()-d.getDay()+w
  const weekDays=()=>Array.from({length:7},(_,i)=>{ const d=new Date(weekStart())
  const days=new Date(year,month+1,0).getDate();
  const firstDW=new Date(year,month,1).getDay();
  const cells=[...Array(firstDW).fill(null),...Array.from({length:days},(_,i)=>i+
  const prevM=()=>{ if(month===0){setYear(y=>y-1);setMonth(11);}else setMonth(m=>
  const nextM=()=>{ if(month===11){setYear(y=>y+1);setMonth(0);}else setMonth(m=>

  const DayCell=({ d, k, isSel, isToday })=>{
    const pct=getDayPct(k);
```

```
  const hasTasks=(tasksByDate[k]||[]).length>0;
  const bgColor = isSel ? C.accent : isToday ? C.accentDim : pct!==null ? compl
  return (
    <button onClick={()=>onSelect(k)} style={{
      textAlign:"center",padding:"7px 2px",borderRadius:12,border:"none",
      background:bgColor,
      color: isSel?"#fff": isToday?C.accent: C.text,
      fontSize:13,fontWeight:isSel||isToday?800:400,
      cursor:"pointer",fontFamily:"inherit",position:"relative",transition:"all
      outline:"none",
    }}>
      {d}
      {/* 100% 달성 별 */}
      {pct===1&&!isSel&&<span style={{ position:"absolute",top:-1,right:0,fontS
    </button>
  );
};

return (
  <div style={{ background:C.surface,borderRadius:20,border:`1px solid ${C.bord
    {/* 헤더 */}
    <div style={{ display:"flex",justifyContent:"space-between",alignItems:"cen
      <button onClick={mode===="month"?prevM:()=>setWOff(w=>w-1)}
        style={{ background:"none",border:"none",color:C.muted,fontSize:22,curs
      <div style={{ display:"flex",alignItems:"center",gap:8 }}>
        <span style={{ fontSize:14,fontWeight:800,color:C.text }}>
          {mode===="month"?`${year}년 ${month+1}월`:(()=>{
            const ws=weekStart(),we=new Date(ws);we.setDate(ws.getDate()+6);
            return `${ws.getMonth()+1}월 ${ws.getDate()}일 ~ ${we.getMonth()+1}월
          })()}
        </span>
        <button onClick={()=>{ setMode(m=>m===="week"?"month":"week"); setWOff(0
          background:C.accentDim,border:"none",borderRadius:8,padding:"3px 10px
          fontSize:10,fontWeight:800,color:C.accent,cursor:"pointer",fontFamily
        }}>{mode===="week"?"월간":"주간"}</button>
      </div>
      <button onClick={mode===="month"?nextM:()=>setWOff(w=>w+1)}
        style={{ background:"none",border:"none",color:C.muted,fontSize:22,curs
    </div>

    {/* 요일 */}
    <div style={{ display:"grid",gridTemplateColumns:"repeat(7,1fr)",padding:"0
      {["일","월","화","수","목","금","토"].map((w,i)=>(
        <div key={w} style={{ textAlign:"center",fontSize:10,color:i===0?C.red:
      ))}
    </div>
```

```
      {/* 날짜 */}
      <div style={{ padding:"4px 12px 14px" }}>
        {mode===="week"?(
          <div style={{ display:"grid",gridTemplateColumns:"repeat(7,1fr)",gap:3
            {weekDays().map(d=>{
              const k=dkey(d);
              return <DayCell key={k} d={d.getDate()} k={k} isSel={k===selDate} i
            })}
          </div>
        ):(
          <div style={{ display:"grid",gridTemplateColumns:"repeat(7,1fr)",gap:3
            {cells.map((d,i)=>{
              if(!d)return<div key={`e${i}`}/>;
              const k=`${year}-${pad(month+1)}-${pad(d)}`;
              return <DayCell key={k} d={d} k={k} isSel={k===selDate} isToday={k=
            })}
          </div>
        )}
      </div>

      {/* 달성률 범례 */}
      <div style={{ display:"flex",alignItems:"center",gap:10,padding:"0 16px 12p
        <span style={{ fontSize:10,color:C.muted }}>달성률:</span>
        {[
          { label:"없음",color:C.border },
          { label:"~33%",color:C.orange+"88" },
          { label:"~66%",color:C.yellow+"88" },
          { label:"~99%",color:C.green+"88" },
          { label:"100%",color:C.green },
        ].map(l=>(
          <div key={l.label} style={{ display:"flex",alignItems:"center",gap:3 }}
            <div style={{ width:10,height:10,borderRadius:3,background:l.color }}
            <span style={{ fontSize:9,color:C.muted }}>{l.label}</span>
          </div>
        ))}
        <span style={{ fontSize:9,color:C.muted }}>⭐=완전달성</span>
      </div>
    </div>
  );
}

/* ═══════════════════════════════════════════════════════════
   TASK ITEM (일정 카드)
═══════════════════════════════════════════════════════════ */
const PALETTE = (C) => [C.accent,"#22c47a","#f5862a","#15b8c8","#e8334a","#e87fac
const CAT_ICONS = { 필수:"⭐", 휴식:"🌿", 루틴:"🔄", 작업:"💼" };
```

```
function TaskCard({ task, idx, C, onToggle, onDelete, onUpdate, onTimerStart }) {
  const [open,setOpen]=useState(false);
  const col=PALETTE(C)[idx%PALETTE(C).length];
  const ac=task.actions||[], dac=ac.filter(a=>a.done).length;
  const catCol = CAT_COLORS(C)[task.category]||col;

  return (
    <div style={{ background:C.card,borderRadius:18,marginBottom:8,
      border:`1px solid ${task.done?C.border:C.borderLit}`,
      borderLeft:`5px solid ${task.done?C.muted:catCol}`,
      opacity:task.done?.58:1,transition:"all .25s",
      boxShadow:task.done?"none":`0 2px 12px ${C.shadow}` }}>

      <div style={{ display:"flex",alignItems:"center",gap:10,padding:"13px 14px"
        {/* 체크 */}
        <button onClick={onToggle} style={{
          width:26,height:26,borderRadius:8,flexShrink:0,
          border:`2px solid ${task.done?catCol:C.borderLit}`,
          background:task.done?catCol:"transparent",
          cursor:"pointer",display:"flex",alignItems:"center",justifyContent:"cen
          fontSize:14,color:"#fff",transition:"all .2s",
        }}>{task.done?"✓":""}</button>

        <div style={{ flex:1,minWidth:0 }}>
          <div style={{ display:"flex",alignItems:"center",gap:6 }}>
            {task.category&&<span style={{ fontSize:12 }}>{CAT_ICONS[task.categor
            <span style={{ fontSize:15,color:C.text,fontWeight:600,
              textDecoration:task.done?"line-through":"none",
              overflow:"hidden",textOverflow:"ellipsis",whiteSpace:"nowrap" }}>{t
          </div>
          {(task.start||task.end)&&(
            <span style={{ fontSize:11,color:C.muted,marginTop:2,display:"block"
              🕐 {task.start||"??"}{task.end?` → ${task.end}`:""}
            </span>
          )}
          {ac.length>0&&(
            <div style={{ display:"flex",alignItems:"center",gap:6,marginTop:4 }}
              <div style={{ height:3,flex:1,background:C.border,borderRadius:2,ov
                <div style={{ height:"100%",borderRadius:2,background:catCol,widt
              </div>
              <span style={{ fontSize:10,color:dac===ac.length?C.green:C.muted,fo
            </div>
          )}
        </div>

        {/* 타이머 */}
        {!task.done&&(
```

```
    <button onClick={()=>onTimerStart(task.text)} title="이 일정으로 타이머 시작'
      style={{ background:C.accentDim,color:C.accent,border:"none",
        borderRadius:10,padding:"6px 10px",fontSize:14,cursor:"pointer",fon
  )}

  {/* 펼치기 */}
  <button onClick={()=>setOpen(o=>!o)}
    style={{ background:"none",border:"none",color:C.muted,cursor:"pointer"
      fontSize:18,padding:"0 2px",flexShrink:0,
      transform:open?"rotate(180deg)":"none",transition:"transform .2s" }}>

  <button onClick={onDelete}
    style={{ background:"none",border:"none",color:C.muted,cursor:"pointer"
</div>

{/* 펼침 */}
{open&&(
  <div style={{ borderTop:`1px solid ${C.border}`,padding:"12px 16px" }}>
    {/* 시간 수정 */}
    <div style={{ display:"grid",gridTemplateColumns:"1fr auto 1fr",alignIt
      <input type="time" value={task.start||""} onChange={e=>onUpdate({star
        style={{ background:C.inputBg,border:`1.5px solid ${C.border}`,bord
          padding:"9px 10px",color:C.text,fontSize:13,fontFamily:"inherit",
      <span style={{ color:C.muted,fontSize:13,textAlign:"center" }}>→</spa
      <input type="time" value={task.end||""} onChange={e=>onUpdate({end:e.
        style={{ background:C.inputBg,border:`1.5px solid ${C.border}`,bord
          padding:"9px 10px",color:C.text,fontSize:13,fontFamily:"inherit",
    </div>

    {/* 행동 단계 */}
    {ac.map((a,i)=>(
      <div key={a.id} style={{ display:"flex",alignItems:"center",gap:8,mar
        <button onClick={()=>onUpdate({actions:ac.map(x=>x.id===a.id?{...x,
          width:18,height:18,borderRadius:5,flexShrink:0,
          border:`2px solid ${a.done?catCol:C.borderLit}`,
          background:a.done?catCol:"transparent",
          cursor:"pointer",display:"flex",alignItems:"center",justifyConten
        }}>{a.done?"✓":""}</button>
        <span style={{ flex:1,fontSize:13,color:a.done?C.muted:C.soft,
          textDecoration:a.done?"line-through":"none",lineHeight:1.5 }}>{a.
      </div>
    ))}

    <button onClick={()=>onUpdate({actions:[...ac,{id:Date.now(),text:"",do
      style={{ width:"100%",background:"none",border:`1.5px dashed ${C.bord
        borderRadius:10,padding:"8px",color:C.muted,fontSize:12,cursor:"poi
      + 행동 단계 추가
```

```
          </button>
        </div>
      )}
    </div>
  );
}

/* ═══════════════════════════════════════════════════════════
   HOME TAB
═══════════════════════════════════════════════════════════ */
const INIT_HABITS=[
  {id:1,name:"물 마시기",emoji:"💧",target:2000,unit:"ml",type:"water",streak:5},
  {id:2,name:"산책",emoji:"🚶",target:30,unit:"분",type:"walk",streak:12},
  {id:3,name:"명상",emoji:"🧘",target:10,unit:"분",type:"med",streak:3},
  {id:4,name:"약 복용",emoji:"💊",target:1,unit:"회",type:"med",streak:8},
];

function HabitSheet({ habit, log, onClose, onSave, C }) {
  const [entries,setE]=useState(log||[]);
  const [val,setV]=useState(habit.type==="water"?200:1);
  const [note,setN]=useState("");
  const total=entries.reduce((s,e)=>s+(e.value||0),0);
  const pct=Math.min(100,(total/habit.target)*100);
  const add=()=>{
    if(!val||val<=0)return;
    const ne=[...entries,{id:Date.now(),value:val,note,time:new Date().toLocaleTi
    setE(ne);onSave(ne);setN("");
  };
  return (
    <Sheet show onClose={onClose} C={C} title={`${habit.emoji} ${habit.name}`}>
      <p style={{ fontSize:12,color:C.muted,margin:"-8px 0 16px" }}>목표 {habit.ta
      {/* 진행 */}
      <div style={{ marginBottom:20 }}>
        <div style={{ display:"flex",justifyContent:"space-between",marginBottom:
          <span style={{ fontSize:12,color:C.muted }}>오늘</span>
          <span style={{ fontSize:15,fontWeight:900,color:pct>=100?C.green:C.acce
        </div>
        <div style={{ height:10,background:C.border,borderRadius:5,overflow:"hidd
          <div style={{ height:"100%",borderRadius:5,background:pct>=100?C.green:
        </div>
        {pct>=100&&<p style={{ textAlign:"center",color:C.green,fontWeight:800,fo
      </div>
      {/* 드래그 */}
      <div style={{ background:C.bg,borderRadius:18,padding:"20px",marginBottom:1
        <p style={{ fontSize:11,color:C.muted,marginBottom:14 }}>☝️ 위아래 드래그로 조
        <DragNum value={val} onChange={setV} min={habit.type==="water"?50:1}
          max={habit.type==="water"?2000:habit.target*3}
```

```
          step={habit.type==="water"?50:1} unit={habit.unit} size={60} color={C.a
        <input value={note} onChange={e=>setN(e.target.value)} placeholder="메모 (
          style={{ marginTop:14,width:"100%",boxSizing:"border-box",background:C.
            border:`1.5px solid ${C.border}`,borderRadius:12,padding:"10px 14px",
            color:C.text,fontSize:13,fontFamily:"inherit",outline:"none" }}/>
        <button onClick={add} style={{ marginTop:10,width:"100%",background:C.acc
          border:"none",borderRadius:14,padding:"13px",fontSize:15,fontWeight:800
          cursor:"pointer",fontFamily:"inherit" }}>기록하기</button>
      </div>
      {entries.map(e=>(
        <div key={e.id} style={{ display:"flex",alignItems:"center",gap:10,paddin
          <span style={{ fontSize:11,color:C.muted,minWidth:44 }}>{e.time}</span>
          <span style={{ fontWeight:800,color:C.text,fontSize:15 }}>{e.value}{hab
          {e.note&&<span style={{ fontSize:12,color:C.muted,flex:1 }}>— {e.note}<
          <button onClick={()=>{ const ne=entries.filter(x=>x.id!==e.id);setE(ne)
            style={{ background:"none",border:"none",color:C.muted,cursor:"pointe
        </div>
      ))}
    </Sheet>
  );
}

function HomeTab({ C, onTimerStart }) {
  const [sub,      setSub]      = useState("tasks");
  const [tasks,    setTasks]    = useState({});
  const [selDate,  setSelDate]  = useState(nowKey());
  const [habits,   setHabits]   = useState(INIT_HABITS);
  const [logs,     setLogs]     = useState({});
  const [selHabit, setSelHabit] = useState(null);
  const [showAdd,  setShowAdd]  = useState(false);
  const [showHAdd, setShowHAdd] = useState(false);
  const [nh,       setNh]       = useState({name:"",emoji:"⭐",target:1,unit:"회",
  const notifTimers = useRef([]);
  const tk=nowKey();
  const dayTasks=tasks[selDate]||[];
  const setDay=fn=>setTasks(prev=>({...prev,[selDate]:typeof fn==="function"?fn(p

  // 오늘 일정 변경시 알림 재스케줄링
  useEffect(() => {
    notifTimers.current.forEach(t => clearTimeout(t));
    const todayTasks = tasks[tk] || [];
    notifTimers.current = scheduleTaskNotifications(todayTasks, tk);
    return () => notifTimers.current.forEach(t => clearTimeout(t));
  }, [tasks, tk]);

  const done=dayTasks.filter(t=>t.done).length;
  const getLog=hid=>logs[`${hid}_${tk}`]||[];
```

```
const getTotal=h=>getLog(h.id).reduce((s,e)=>s+(e.value||0),0);
const isDone=h=>getTotal(h)>=h.target;
const doneCnt=habits.filter(h=>isDone(h)).length;

return (
  <div>
    {selHabit&&<HabitSheet habit={selHabit} log={getLog(selHabit.id)} C={C}
      onClose={()=>setSelHabit(null)} onSave={e=>setLogs(p=>({...p,[`${selHabit

    <AddTaskSheet show={showAdd} onClose={()=>setShowAdd(false)} C={C} defaultD
      onAdd={t=>{ setDay(prev=>[...prev,t].sort((a,b)=>(a.start||"").localeComp

    {/* 알림 권한 요청 배너 */}
    <NotifPermBanner C={C} onGrant={() => {
      // 권한 허용 후 즉시 오늘 일정 재스케줄
      const todayTasks = tasks[tk] || [];
      notifTimers.current.forEach(t => clearTimeout(t));
      notifTimers.current = scheduleTaskNotifications(todayTasks, tk);
      toastBus.emit({ id:Date.now(), icon:"🔔", title:"알림 설정 완료!", body:"일정
    }}/>

    {/* 달력 */}
    <CalendarView tasksByDate={tasks} selDate={selDate} onSelect={setSelDate} C

    {/* 서브탭 */}
    <div style={{ display:"flex",background:C.surface,borderRadius:14,border:`1
      padding:4,gap:4,marginBottom:16 }}>
      {[
        {id:"tasks", icon:"📋", label:`일정${dayTasks.length>0?` (${done}/${dayT
        {id:"habits",icon:"🔥", label:`습관 (${doneCnt}/${habits.length})`},
      ].map(t=>(
        <button key={t.id} onClick={()=>setSub(t.id)} style={{
          flex:1,padding:"10px 8px",borderRadius:10,border:"none",cursor:"point
          background:sub===t.id?C.accent:"transparent",
          color:sub===t.id?"#fff":C.muted,
          fontSize:13,fontWeight:700,transition:"all .2s",
          display:"flex",alignItems:"center",justifyContent:"center",gap:6,
        }}>
          {t.icon} {t.label}
        </button>
      ))}
    </div>

    {/* 일정 */}
    {sub==="tasks"&&(
      <div>
        <div style={{ display:"flex",justifyContent:"space-between",alignItems:
```

```
        <span style={{ fontSize:13,fontWeight:800,color:C.text }}>
          {new Date(selDate+"T00:00:00").toLocaleDateString("ko-KR",{month:"s
          {selDate===tk&&<span style={{ marginLeft:6,fontSize:11,background:C
        </span>
        {dayTasks.length>0&&<span style={{ fontSize:11,color:C.muted }}>{done
      </div>
      {dayTasks.length>0&&(
        <div style={{ height:5,background:C.border,borderRadius:3,overflow:"h
          <div style={{ height:"100%",borderRadius:3,
            background:`linear-gradient(90deg,${C.accent},${C.green})`,
            width:`${(done/dayTasks.length)*100}%`,transition:"width .5s" }}/
        </div>
      )}
      {dayTasks.length===0&&(
        <div style={{ textAlign:"center",color:C.muted,padding:"32px 0",fontS
          이 날 일정이 없어요 ✨<br/>
          <span style={{ fontSize:12 }}>아래 + 버튼으로 추가하세요</span>
        </div>
      )}
      {dayTasks.map((t,i)=>(
        <TaskCard key={t.id} task={t} idx={i} C={C}
          onToggle={()=>setDay(prev=>prev.map(x=>x.id===t.id?{...x,done:!x.do
          onDelete={()=>setDay(prev=>prev.filter(x=>x.id!==t.id))}
          onUpdate={patch=>setDay(prev=>prev.map(x=>x.id===t.id?{...x,...patc
          onTimerStart={label=>onTimerStart(label)}/>
      ))}
      {/* FAB */}
      <div style={{ marginTop:16 }}>
        <button onClick={()=>setShowAdd(true)} style={{
          width:"100%",background:C.accent,color:"#fff",border:"none",borderR
          padding:"15px",fontSize:15,fontWeight:900,cursor:"pointer",fontFami
          boxShadow:`0 4px 20px ${C.accentGlow}`,
          display:"flex",alignItems:"center",justifyContent:"center",gap:8,
        }}>
          <span style={{ fontSize:20 }}>+</span> 일정 추가
        </button>
      </div>
    </div>
  )}

  {/* 습관 */}
  {sub==="habits"&&(
    <div>
      <div style={{ background:C.surface,borderRadius:14,padding:"12px 16px",
        border:`1px solid ${C.border}`,marginBottom:14 }}>
        <div style={{ display:"flex",justifyContent:"space-between",alignItem
          <span style={{ fontSize:12,color:C.muted,fontWeight:600 }}>오늘 달성<
```

```
    <span style={{ fontSize:18,fontWeight:900,color:doneCnt===habits.le
      {doneCnt}/{habits.length}{doneCnt===habits.length&&habits.length>
    </span>
  </div>
  <div style={{ height:6,background:C.border,borderRadius:3,overflow:"h
    <div style={{ height:"100%",borderRadius:3,background:`linear-gradi
      width:`${habits.length?(doneCnt/habits.length)*100:0}%`,transitio
  </div>
</div>
{habits.map(h=>{
  const total=getTotal(h),pct=Math.min(100,(total/h.target)*100),done=i
  return (
    <div key={h.id} onClick={()=>setSelHabit(h)} style={{
      background:done?C.greenDim:C.card,borderRadius:16,marginBottom:8,
      border:`1px solid ${done?C.green:C.borderLit}`,
      padding:"14px 16px",cursor:"pointer",transition:"all .2s",
      boxShadow:done?"none":`0 2px 10px ${C.shadow}`,
    }}>
      <div style={{ display:"flex",alignItems:"center",gap:12 }}>
        <div style={{ width:48,height:48,borderRadius:14,flexShrink:0,
          background:done?C.green:C.inputBg,border:`2px solid ${done?C.
          display:"flex",alignItems:"center",justifyContent:"center",fo
          {done?"✓":h.emoji}
        </div>
        <div style={{ flex:1,minWidth:0 }}>
          <div style={{ display:"flex",justifyContent:"space-between",a
            <p style={{ margin:0,fontSize:15,fontWeight:700,color:C.tex
            <span style={{ fontSize:13,fontWeight:800,color:done?C.gree
              {total}<span style={{ fontSize:10,color:C.muted,fontWeigh
            </span>
          </div>
          <div style={{ height:4,background:C.border,borderRadius:2,mar
            <div style={{ height:"100%",borderRadius:2,background:done?
              width:`${pct}%`,transition:"width .5s" }}/>
          </div>
          <div style={{ display:"flex",justifyContent:"space-between",m
            <span style={{ fontSize:11,color:C.muted }}>🔥 {h.streak}일
            <span style={{ fontSize:11,color:C.muted }}>탭하여 기록 →</spa
          </div>
        </div>
      </div>
    </div>
  );
})}
{!showHAdd?(
  <button onClick={()=>setShowHAdd(true)} style={{ width:"100%",backgro
    border:`1.5px dashed ${C.border}`,borderRadius:14,padding:"12px",
```

```
              color:C.muted,fontSize:13,cursor:"pointer",fontFamily:"inherit" }}>
          ):(
            <div style={{ background:C.card,borderRadius:16,padding:16,border:`1p
              <div style={{ display:"flex",gap:8,marginBottom:10 }}>
                <input value={nh.emoji} onChange={e=>setNh(h=>({...h,emoji:e.targ
                  style={{ width:52,background:C.inputBg,border:`1.5px solid ${C.
                    padding:"8px 4px",color:C.text,fontSize:22,textAlign:"center"
                <input value={nh.name} onChange={e=>setNh(h=>({...h,name:e.target
                  style={{ flex:1,background:C.inputBg,border:`1.5px solid ${C.bo
                    padding:"8px 12px",color:C.text,fontSize:14,fontFamily:"inher
              </div>
              <div style={{ display:"flex",gap:8,marginBottom:12 }}>
                <input type="number" value={nh.target} onChange={e=>setNh(h=>({..
                  style={{ width:90,background:C.inputBg,border:`1.5px solid ${C.
                    padding:"8px 10px",color:C.text,fontSize:14,fontFamily:"inher
                <input value={nh.unit} onChange={e=>setNh(h=>({...h,unit:e.target
                  style={{ flex:1,background:C.inputBg,border:`1.5px solid ${C.bo
                    padding:"8px 12px",color:C.text,fontSize:14,fontFamily:"inher
              </div>
              <div style={{ display:"flex",gap:8 }}>
                <button onClick={()=>setShowHAdd(false)} style={{ flex:1,backgrou
                  border:`1.5px solid ${C.border}`,borderRadius:12,padding:"10px"
                <button onClick={()=>{ if(!nh.name.trim())return; setHabits(p=>[.
                  style={{ flex:2,background:C.accent,color:"#fff",border:"none",
              </div>
            </div>
          )}
        </div>
      )}
    </div>
  );
}

/* ============================================================
   TIMER TAB
============================================================ */
const PH={IDLE:"idle",RUN:"run",PAUSED:"paused",ENDED:"ended"};
function nextSugg(n){ return n===0?30*60:n===1?60*60:25*60; }

function TimerTab({ C, initTask }) {
  const [phase,  setPhase] =useState(PH.IDLE);
  const [sec,    setSec]   =useState(5*60);
  const [total,  setTotal] =useState(5*60);
  const [sess,   setSess]  =useState(0);
  const [lock,   setLock]  =useState(false);
  const [selP,   setSelP]  =useState(0);
  const [cMin,   setCMin]  =useState(5);
```

```
const [cSec,   setCSec]  =useState(0);
const [showC,  setShowC] =useState(false);
const [task,   setTask]  =useState(initTask||null);
const ref=useRef(null);
const PRESETS=[{l:"5분",i:"⚡",s:5*60},{l:"30분",i:"🎯",s:30*60},{l:"1시간",i:"🔥
const chosen=selP<3?PRESETS[selP].s:cMin*60+cSec;
useEffect(()=>{ if(initTask)setTask(initTask); },[initTask]);
useEffect(()=>{
  if(phase===PH.RUN){ ref.current=setInterval(()=>setSec(s=>{ if(s<=1){clearInt
  else clearInterval(ref.current);
  return()=>clearInterval(ref.current);
},[phase]);
const doStart=(s=null)=>{ const t=s??chosen; setTotal(t);setSec(t);setPhase(PH.
const stop=()=>{ setPhase(PH.IDLE);setLock(false);setSec(chosen); };
const add5=()=>{ setTotal(t=>t+300);setSec(s=>s+300);setPhase(PH.RUN); };
const pct=total>0?Math.max(0,(total-sec)/total):0;
const R=108,circ=2*Math.PI*R;

if(lock&&(phase===PH.RUN||phase===PH.ENDED)){
  return (
    <div style={{ position:"fixed",inset:0,zIndex:400,background:C.isDark?"#070
      display:"flex",flexDirection:"column",alignItems:"center",justifyContent:
      <style>{`@keyframes bglow{0%,100%{opacity:.04}50%{opacity:.15}}`}</style>
      <div style={{ position:"absolute",inset:0,pointerEvents:"none",
        background:`radial-gradient(circle at 50% 42%, ${C.accent}22 0%,transpa
        animation:"bglow 3.5s ease-in-out infinite" }}/>
      <p style={{ fontSize:11,color:C.muted,letterSpacing:3,textTransform:"uppe
      {task&&<p style={{ fontSize:16,color:C.soft,fontWeight:600,margin:0,textA
      <div style={{ position:"relative",width:264,height:264 }}>
        <svg width="264" height="264" style={{ transform:"rotate(-90deg)" }}>
          <circle cx="132" cy="132" r={R} fill="none" stroke={C.border} strokeW
          <circle cx="132" cy="132" r={R} fill="none" stroke={C.accent} strokeW
            strokeLinecap="round" strokeDasharray={`${circ*pct} ${circ}`}
            style={{ transition:"stroke-dasharray 1s linear",filter:`drop-shado
        </svg>
        <div style={{ position:"absolute",inset:0,display:"flex",flexDirection:
          {phase===PH.ENDED?(
            <><span style={{ fontSize:40 }}>🎉</span><span style={{ fontSize:16
          ):(
            <><span style={{ fontSize:58,fontWeight:900,fontFamily:"'DM Mono','
          )}
        </div>
      </div>
      {phase===PH.ENDED?(
        <div style={{ display:"flex",flexDirection:"column",gap:10,width:"100%"
          <button onClick={()=>doStart(nextSugg(sess-1))} style={{ background:C
          <div style={{ display:"flex",gap:8 }}>
```

```
            <button onClick={add5} style={{ flex:1,background:C.card,color:C.so
            <button onClick={stop} style={{ flex:1,background:C.redDim,color:C.
          </div>
        </div>
      ):(
        <div style={{ position:"absolute",bottom:48,display:"flex",flexDirectio
          <button onClick={()=>setLock(false)} style={{ background:"none",borde
        </div>
      )}
    </div>
  );
}

return (
  <div style={{ display:"flex",flexDirection:"column",alignItems:"center",gap:2
    {task&&phase!==PH.IDLE&&(
      <div style={{ background:C.accentDim,borderRadius:12,padding:"7px 16px",f
        📌 {task}<button onClick={()=>setTask(null)} style={{ background:"none"
      </div>
    )}

    {/* 프리셋 */}
    {phase===PH.IDLE&&(
      <div style={{ display:"flex",gap:8,width:"100%" }}>
        {PRESETS.map((p,i)=>(
          <button key={i} onClick={()=>{ setSelP(i);setShowC(false); }} style={
            flex:1,padding:"12px 6px",borderRadius:16,
            background:selP===i&&!showC?C.accent:C.card,
            color:selP===i&&!showC?"#fff":C.muted,
            border:`1.5px solid ${selP===i&&!showC?C.accent:C.border}`,
            fontSize:12,fontWeight:800,cursor:"pointer",fontFamily:"inherit",tr
            display:"flex",flexDirection:"column",alignItems:"center",gap:4,
          }}><span style={{ fontSize:20 }}>{p.i}</span>{p.l}</button>
        ))}
        <button onClick={()=>{ setSelP(3);setShowC(true); }} style={{
          flex:1,padding:"12px 6px",borderRadius:16,
          background:showC?C.accent:C.card,color:showC?"#fff":C.muted,
          border:`1.5px solid ${showC?C.accent:C.border}`,
          fontSize:12,fontWeight:800,cursor:"pointer",fontFamily:"inherit",tran
          display:"flex",flexDirection:"column",alignItems:"center",gap:4,
        }}><span style={{ fontSize:20 }}>✏️</span>직접</button>
      </div>
    )}

    {/* 직접 설정 */}
    {phase===PH.IDLE&&showC&&(
      <div style={{ background:C.surface,borderRadius:20,padding:"22px 32px",bo
```

```
    <p style={{ textAlign:"center",fontSize:11,color:C.muted,marginBottom:1
    <div style={{ display:"flex",justifyContent:"center",alignItems:"flex-s
      <DragNum value={cMin} onChange={setCMin} min={0} max={180} step={1} u
      <span style={{ fontSize:52,fontWeight:900,color:C.muted,lineHeight:1,
      <DragNum value={cSec} onChange={setCSec} min={0} max={59} step={5} un
    </div>
  </div>
)}

{/* 원형 타이머 */}
<div style={{ position:"relative",width:256,height:256 }}>
  <svg width="256" height="256" style={{ transform:"rotate(-90deg)" }}>
    <defs>
      <linearGradient id="tg" x1="0%" y1="0%" x2="100%" y2="100%">
        <stop offset="0%" stopColor={C.accent}/><stop offset="100%" stopCol
      </linearGradient>
      <filter id="gf"><feGaussianBlur stdDeviation="3" result="b"/><feMerge
    </defs>
    <circle cx="128" cy="128" r={R} fill="none" stroke={C.border} strokeWid
    <circle cx="128" cy="128" r={R} fill="none" stroke="url(#tg)" strokeWid
      strokeLinecap="round" strokeDasharray={`${circ*pct} ${circ}`} filter=
      style={{ transition:phase===PH.RUN?"stroke-dasharray 1s linear":"stro
  </svg>
  <div style={{ position:"absolute",inset:0,display:"flex",flexDirection:"c
    {phase===PH.IDLE?(
      <><span style={{ fontSize:52,fontWeight:900,fontFamily:"'DM Mono','Co
    ):phase===PH.ENDED?(
      <><span style={{ fontSize:38 }}>🎉</span><span style={{ fontSize:14,c
    ):(
      <><span style={{ fontSize:52,fontWeight:900,fontFamily:"'DM Mono','Co
    )}
  </div>
</div>

{/* 세션 점 */}
{sess>0&&<div style={{ display:"flex",gap:5 }}>{Array.from({length:Math.min
  <div key={i} style={{ width:8,height:8,borderRadius:"50%",background:C.ac
))}{sess>8&&<span style={{ fontSize:11,color:C.muted }}>+{sess-8}</span>}</

{/* 버튼 */}
{phase===PH.IDLE&&<button onClick={()=>doStart()} style={{ background:C.acc
{(phase===PH.RUN||phase===PH.PAUSED)&&(
  <div style={{ display:"flex",gap:8,width:"100%" }}>
    <button onClick={()=>setPhase(phase===PH.RUN?PH.PAUSED:PH.RUN)} style={
    <button onClick={()=>setLock(true)} style={{ flex:1,background:C.card,c
    <button onClick={stop} style={{ flex:1,background:C.redDim,color:C.red,
  </div>
```

```
      )}
      {phase===PH.ENDED&&(
        <div style={{ display:"flex",flexDirection:"column",gap:10,width:"100%" }
          <button onClick={()=>doStart(nextSugg(sess-1))} style={{ background:C.a
          <div style={{ display:"flex",gap:8 }}>
            <button onClick={add5} style={{ flex:1,background:C.card,color:C.soft
            <button onClick={stop} style={{ flex:1,background:C.redDim,color:C.re
          </div>
        </div>
      )}
    </div>
  );
}

/* ═══════════════════════════════════════════════════════════
   MIND TAB  — 일기 형식
═══════════════════════════════════════════════════════════ */
const EMOTIONS_LIST=[
  {id:"calm",l:"차분",e:"😌",c:"#15b8c8"},
  {id:"happy",l:"기쁨",e:"😊",c:"#22c47a"},
  {id:"anxious",l:"불안",e:"😰",c:"#d4a000"},
  {id:"frustrated",l:"짜증",e:"😤",c:"#f5862a"},
  {id:"sad",l:"슬픔",e:"😔",c:"#8899ff"},
  {id:"overwhelmed",l:"압도",e:"😵",c:"#e8334a"},
  {id:"focused",l:"집중",e:"🎯",c:"#6c5fff"},
  {id:"tired",l:"피로",e:"😴",c:"#9c98b8"},
];

function MindTab({ C }) {
  const [diaryByDate, setDiaryByDate] = useState({});
  const [selDate, setSelDate] = useState(nowKey());
  const [thought, setThought] = useState("");
  const [emos,    setEmos]    = useState([]);
  const [energy,  setEnergy]  = useState(null);
  const [reframe, setReframe] = useState("");
  const [loading, setLoading] = useState(false);
  const [showEnergy, setShowEnergy] = useState(false);

  const tk = nowKey();
  const entries = diaryByDate[selDate] || [];
  const todayEntries = diaryByDate[tk] || [];

  // 날짜 목록 (기록 있는 날)
  const datesWithEntries = Object.keys(diaryByDate).filter(k => diaryByDate[k]?.l

  const getReframe = async () => {
    if (!thought.trim()) return;
```

```
  setLoading(true);
  try {
    const res = await fetch("https://api.anthropic.com/v1/messages", {
      method:"POST", headers:{"Content-Type":"application/json"},
      body:JSON.stringify({
        model:"claude-sonnet-4-20250514", max_tokens:350,
        system:"ADHD 전문 CBT 코치. 부정적 자동적 사고를 인지 재구성으로 균형 잡힌 시각 3문장 C
        messages:[{role:"user",content:`"${thought}" — CBT 인지 재구성`}],
      }),
    });
    const d = await res.json();
    setReframe(d.content?.[0]?.text||"");
  } catch { setReframe("잠시 후 다시 시도해주세요."); }
  setLoading(false);
};

const save = () => {
  if (!thought.trim() && emos.length===0 && !energy) return;
  const entry = {
    id: Date.now(),
    time: new Date().toLocaleTimeString("ko-KR",{hour:"2-digit",minute:"2-digit
    thought, reframe, emos, energy,
  };
  setDiaryByDate(prev => ({ ...prev, [selDate]: [...(prev[selDate]||[]), entry]
  setThought(""); setReframe(""); setEmos([]); setEnergy(null); setShowEnergy(f
};

// 날짜 레이블
const dateLabel = k => {
  if (k === tk) return "오늘";
  const d = new Date(k+"T00:00:00");
  return d.toLocaleDateString("ko-KR",{month:"short",day:"numeric",weekday:"sho
};

return (
  <div style={{ display:"flex",flexDirection:"column",gap:14 }}>

    {/* 날짜 선택 탭 (기록 있는 날 + 오늘) */}
    <div style={{ display:"flex",gap:6,overflowX:"auto",paddingBottom:4,scrollb
      {[tk,...datesWithEntries.filter(d=>d!==tk)].slice(0,10).map(k=>(
        <button key={k} onClick={()=>setSelDate(k)} style={{
          flexShrink:0,padding:"7px 14px",borderRadius:20,
          background:selDate===k?C.accent:C.card,
          color:selDate===k?"#fff":C.muted,
          border:`1.5px solid ${selDate===k?C.accent:C.border}`,
          fontSize:12,fontWeight:700,cursor:"pointer",fontFamily:"inherit",tran
        }}>{dateLabel(k)}</button>
```

```
  ))}
</div>

{/* 입력 (오늘만) */}
{selDate === tk && (
  <div style={{ background:C.surface,borderRadius:20,padding:20,border:`1px
    <p style={{ fontSize:15,fontWeight:800,color:C.text,marginBottom:4 }}>
    <p style={{ fontSize:12,color:C.muted,marginBottom:14,lineHeight:1.6 }}
      무슨 생각이든 그냥 적어보세요. 판단하지 않아도 돼요.
    </p>

    <textarea value={thought} onChange={e=>setThought(e.target.value)}
      placeholder="지금 드는 생각, 걱정, 감정을 자유롭게..."
      rows={4} style={{ width:"100%",boxSizing:"border-box",background:C.in
        border:`1.5px solid ${C.border}`,borderRadius:14,padding:"12px 14px
        color:C.text,fontSize:14,fontFamily:"inherit",outline:"none",resize
        lineHeight:1.7,marginBottom:12 }}/>

    {/* 감정 */}
    <div style={{ display:"flex",flexWrap:"wrap",gap:6,marginBottom:14 }}>
      {EMOTIONS_LIST.map(e=>(
        <button key={e.id} onClick={()=>setEmos(p=>p.includes(e.id)?p.filte
          padding:"6px 12px",borderRadius:20,cursor:"pointer",fontFamily:"i
          border:`1.5px solid ${emos.includes(e.id)?e.c:C.border}`,
          background:emos.includes(e.id)?e.c+"25":C.card,
          color:emos.includes(e.id)?e.c:C.muted,
          fontSize:12,fontWeight:600,transition:"all .15s",
        }}>{e.e} {e.l}</button>
      ))}
    </div>

    {/* CBT 재구성 */}
    {thought.trim()&&(
      <button onClick={getReframe} disabled={loading} style={{
        width:"100%",background:loading?C.border:C.accent,color:loading?C.m
        border:"none",borderRadius:14,padding:"12px",fontSize:13,fontWeight
        cursor:"pointer",fontFamily:"inherit",transition:"all .2s",marginBo
      }}>{loading?"✨ 분석 중...":"✨ 다른 관점 찾기 (CBT)"}</button>
    )}

    {reframe&&(
      <div style={{ background:C.accentDim,borderRadius:14,padding:"14px 16
        <p style={{ fontSize:11,color:C.accent,fontWeight:800,marginBottom:
        <p style={{ fontSize:13,color:C.soft,margin:0,lineHeight:1.8 }}>{re
      </div>
    )}
```

```
    {/* 에너지 (접힘) */}
    <div style={{ background:C.bg,borderRadius:14,overflow:"hidden",marginB
      <button onClick={()=>setShowEnergy(e=>!e)} style={{
        width:"100%",display:"flex",justifyContent:"space-between",alignIte
        padding:"12px 14px",background:"none",border:"none",cursor:"pointer
      }}>
        <span style={{ fontSize:12,fontWeight:700,color:C.muted }}>⚡ 에너지
        <span style={{ fontSize:16,color:C.muted,transform:showEnergy?"rota
      </button>
      {showEnergy&&(
        <div style={{ padding:"0 14px 14px" }}>
          <div style={{ display:"flex",gap:6 }}>
            {[{v:1,e:"😴"},{v:2,e:"😐"},{v:3,e:"⚡"},{v:4,e:"🔥"},{v:5,e:"
              <button key={v} onClick={()=>setEnergy(v)} style={{
                flex:1,padding:"10px 4px",borderRadius:12,border:"none",cur
                background:energy===v?C.accent:C.card,color:energy===v?"#ff
                display:"flex",flexDirection:"column",alignItems:"center",g
                outline:energy===v?`2px solid ${C.accent}`:"none",transitio
              }}><span style={{ fontSize:18 }}>{e}</span><span style={{ fon
            ))}
          </div>
        </div>
      )}
    </div>

    <button onClick={save} disabled={!thought.trim()&&emos.length===0&&!ene
      width:"100%",background:C.accent,color:"#fff",border:"none",borderRad
      padding:"13px",fontSize:14,fontWeight:800,cursor:"pointer",fontFamily
      boxShadow:`0 4px 16px ${C.accentGlow}`,
      opacity:(!thought.trim()&&emos.length===0&&!energy)?.4:1,
    }}>💾 일기에 저장</button>
  </div>
)}

{/* 일기 목록 */}
{entries.length > 0 ? (
  <div>
    <p style={{ fontSize:12,fontWeight:700,color:C.muted,marginBottom:10,le
      {dateLabel(selDate)}의 기록 · {entries.length}개
    </p>
    {entries.map((e,i)=>(
      <div key={e.id} style={{ background:C.surface,borderRadius:18,marginB
        border:`1px solid ${C.border}`,padding:"16px 18px",
        boxShadow:`0 2px 10px ${C.shadow}` }}>
        {/* 헤더 */}
        <div style={{ display:"flex",justifyContent:"space-between",alignIt
          <div style={{ display:"flex",alignItems:"center",gap:8 }}>
```

```
                <div style={{ width:8,height:8,borderRadius:"50%",background:C.
                <span style={{ fontSize:12,color:C.muted,fontWeight:600 }}>{e.t
                <span style={{ fontSize:11,color:C.muted }}>#{i+1}</span>
              </div>
              {e.energy&&<span style={{ fontSize:11,background:C.accentDim,colo
                padding:"2px 8px",borderRadius:8,fontWeight:700 }}>에너지 {e.ener
            </div>

            {/* 감정 태그 */}
            {e.emos?.length>0&&(
              <div style={{ display:"flex",flexWrap:"wrap",gap:4,marginBottom:1
                {e.emos.map(eid=>{
                  const em=EMOTIONS_LIST.find(x=>x.id===eid);
                  return em?<span key={eid} style={{ fontSize:11,padding:"3px 1
                    background:em.c+"22",color:em.c,fontWeight:600 }}>{em.e} {e
                })}
              </div>
            )}

            {/* 생각 */}
            {e.thought&&(
              <div style={{ background:C.inputBg,borderRadius:12,padding:"12px
                <p style={{ fontSize:14,color:C.text,margin:0,lineHeight:1.8,fo
                  "{e.thought}"
                </p>
              </div>
            )}

            {/* CBT 재구성 */}
            {e.reframe&&(
              <div style={{ background:C.accentDim,borderRadius:10,padding:"10p
                border:`1px solid ${C.accent}33` }}>
                <p style={{ fontSize:10,color:C.accent,fontWeight:800,marginBot
                <p style={{ fontSize:12,color:C.soft,margin:0,lineHeight:1.7 }}
              </div>
            )}
          </div>
        ))}
      </div>
    ) : selDate !== tk && (
      <div style={{ textAlign:"center",color:C.muted,padding:"40px 0",fontSize:
        이 날의 기록이 없어요 📖
      </div>
    )}
  </div>
);
}
```

```
/* ═══════════════════════════════════════════════════════════
   ROOT
═══════════════════════════════════════════════════════════ */
const TABS=[
  {id:"home", icon:"🏠",label:"홈"},
  {id:"timer",icon:"⏱️",label:"타이머"},
  {id:"mind", icon:"📖",label:"일기"},
];

export default function App() {
  const [tab,       setTab]      = useState("home");
  const [dark,      setDark]     = useState(false);  // 기본 라이트
  const [timerTask, setTimerTask]= useState(null);
  const C = dark ? DARK : LIGHT;

  const goTimer = label => { setTimerTask(label); setTab("timer"); };

  return (
    <div style={{ minHeight:"100vh",background:C.bg,color:C.text,
      fontFamily:"'Pretendard','Apple SD Gothic Neo','Noto Sans KR',sans-serif",
      transition:"background .3s,color .3s" }}>
      <style>{`
        *{box-sizing:border-box;-webkit-tap-highlight-color:transparent;}
        input,textarea{outline:none;}
        ::-webkit-scrollbar{width:0;height:0;}
        input[type=time]::-webkit-calendar-picker-indicator{filter:${dark?"invert
        @keyframes fadeUp{from{opacity:0;transform:translateY(12px)}to{opacity:1;
      `}</style>

      {/* 인앱 토스트 알림 */}
      <ToastBanner C={C}/>

      {/* 헤더 */}
      <div style={{ position:"sticky",top:0,zIndex:100,
        background:C.navBg,backdropFilter:"blur(18px)",
        borderBottom:`1px solid ${C.border}` }}>
        <div style={{ maxWidth:480,margin:"0 auto",padding:"14px 20px",
          display:"flex",justifyContent:"space-between",alignItems:"center" }}>
          <div>
            <h1 style={{ margin:0,fontSize:20,fontWeight:900,letterSpacing:-.5 }}
              <span style={{ color:C.accent }}>Focus</span>Flow
            </h1>
            <p style={{ margin:0,fontSize:11,color:C.muted }}>
              {new Date().toLocaleDateString("ko-KR",{month:"long",day:"numeric",
            </p>
          </div>
```

```
          <button onClick={()=>setDark(d=>!d)} style={{
            background:C.surface,border:`1.5px solid ${C.border}`,
            borderRadius:24,padding:"8px 14px",
            display:"flex",alignItems:"center",gap:6,
            cursor:"pointer",fontFamily:"inherit",transition:"all .2s",
          }}>
            <span style={{ fontSize:15 }}>{dark?"☀️":"🌙"}</span>
            <span style={{ fontSize:11,fontWeight:700,color:C.muted }}>{dark?"라이
          </button>
        </div>
      </div>

      {/* 콘텐츠 */}
      <div style={{ maxWidth:480,margin:"0 auto",padding:"18px 16px 110px",
        animation:"fadeUp .25s ease" }} key={tab}>
        {tab==="home"  && <HomeTab  C={C} onTimerStart={goTimer}/>}
        {tab==="timer" && <TimerTab C={C} initTask={timerTask}/>}
        {tab==="mind"  && <MindTab  C={C}/>}
      </div>

      {/* 하단 탭바 */}
      <div style={{ position:"fixed",bottom:0,left:0,right:0,zIndex:50,
        background:C.navBg,backdropFilter:"blur(20px)",
        borderTop:`1px solid ${C.border}`,
        paddingBottom:"env(safe-area-inset-bottom)" }}>
        <div style={{ maxWidth:480,margin:"0 auto",display:"flex",padding:"6px 8p
          {TABS.map(t=>(
            <button key={t.id} onClick={()=>setTab(t.id)} style={{
              flex:1,display:"flex",flexDirection:"column",alignItems:"center",
              gap:3,padding:"6px 4px 8px",border:"none",background:"none",
              cursor:"pointer",fontFamily:"inherit",transition:"all .2s",
            }}>
              <div style={{ width:46,height:30,borderRadius:12,
                background:tab===t.id?C.accentDim:"transparent",
                display:"flex",alignItems:"center",justifyContent:"center",
                fontSize:18,transition:"all .2s" }}>{t.icon}</div>
              <span style={{ fontSize:10,fontWeight:700,
                color:tab===t.id?C.accent:C.muted,transition:"color .2s" }}>{t.la
            </button>
          ))}
        </div>
      </div>
    </div>
  );
}
```